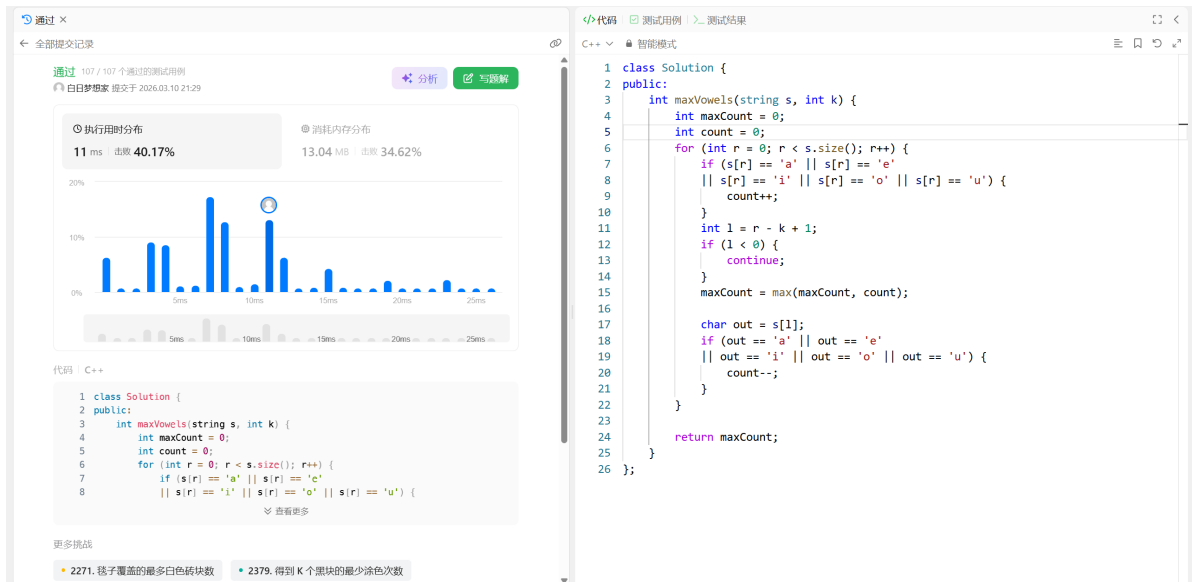


2000.反转单词前缀

The screenshot shows a LeetCode problem page for '2000. 反转单词前缀'. On the left, the '通过' (Accepted) status is shown with a bar chart indicating 100% success rate. The execution time is 0ms and memory usage is 8.33 MB. The right side displays the C++ code for the solution.

```
1 class Solution {
2 public:
3     string reversePrefix(string word, char ch) {
4         int left = 0, right = -1;
5         for (int i = 0; i < word.size(); i++) {
6             if (word[i] == ch) {
7                 right = i;
8             }
9             if (right != -1) {
10                break;
11            }
12        }
13
14        while (left < right) {
15            char temp = word[left];
16            word[left] = word[right];
17            word[right] = temp;
18            left++;
19            right--;
20        }
21
22        return word;
23    }
24};
```

```
1  class Solution {
2  public:
3      string reversePrefix(string word, char ch) {
4          int left = 0; //左指针始终为0
5          int right = -1; //右指针初始化为-1
6          for (int i = 0; i < word.size(); i++) { //遍历字符串，直至找到目标字符，将
              右指针更新
7              if (word[i] == ch) {
8                  right = i;
9              }
10             if (right != -1) { //找到第一个目标字符直接跳出整个循环，符合题目要求
11                 break;
12             }
13         }
14
15         while (left < right) { //交换左右指针指向元素，达到反转单词前缀的目的
16             char temp = word[left];
17             word[left] = word[right];
18             word[right] = temp;
19             left++;
20             right--;
21         }
22
23         return word;
24     }
25};
```



1456.定长字串中元音的最大数目

```
1 class Solution {
2 public:
3     int maxVowels(string s, int k) {
4         int maxCount = 0; //初始化元音字符最大数目为0
5         int count = 0; //初始化当前窗口中元音字符数目为0
6         for (int r = 0; r < s.size(); r++) {
7             if (s[r] == 'a' || s[r] == 'e'
8                 || s[r] == 'i' || s[r] == 'o' || s[r] == 'u') {
9                 count++;
10            }
11            int l = r - k + 1; //当前窗口左端点下标固定表达公式（r为右端点下标，k为窗口固定长度）
12            if (l < 0) { //表明定长窗口还未形成，右端点继续向右滑动
13                continue;
14            }
15            maxCount = max(maxCount, count); //定长窗口形成，更新元音字符最大数目
16
17            char out = s[l]; //左端点离开窗口（定长窗口向右滑动）
18            if (out == 'a' || out == 'e'
19                || out == 'i' || out == 'o' || out == 'u') {
20                count--;
21            }
22        }
23
24        return maxCount;
25    }
26};
```